

ME315A_LAB03

Letícia Santos Tavares

HANDS-ON

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.

- Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;

– Para o arquivo de vôos:

- a. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`
- b. Leia apenas 1 milhão de linhas por vez;
- c. Remova vôos em que o aeroporto de destino comece com a letra 1;
- d. Remova registros em que existam pelo menos uma coluna faltante;
- e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
- f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Importando airports.csv:

```
airports <- read_csv("airports.csv") |>
  select(c("IATA_CODE", "CITY", "STATE", "LATITUDE", "LONGITUDE"))
```

Rows: 322 Columns: 7

-- Column specification -----

Delimiter: ","

chr (5): IATA_CODE, AIRPORT, CITY, STATE, COUNTRY

dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Importando airlines.csv:

```
airlines <- read_csv("airlines.csv")
```

Rows: 14 Columns: 2

-- Column specification -----

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Importando flights.csv por partes:

```
flights <- read_csv_chunked(
  "flights.csv",
  DataFrameCallback$new(\(x, pos) {
    x |>
      select(DESTINATION_AIRPORT, ARRIVAL_DELAY) |>
      filter(!str_starts(DESTINATION_AIRPORT, "1")) |>
      drop_na() |>
      group_by(DESTINATION_AIRPORT) |>
      summarise(
        soma = sum(ARRIVAL_DELAY),
        n_cia = n()
      )
  })
```

```

    }),
    chunk_size = 1000000
  )

```

```

-- Column specification -----
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),
  CANCELLATION_REASON = col_character()
)
i Use `spec()` for the full column specifications.

```

```

#média de voos atrasados por aeroporto de destino:
flights_mean<- flights |>
  group_by(DESTINATION_AIRPORT) |>
  summarise(
    MEAN_DELAY = sum(soma) / sum(n_cia)
  )

```

2. Selecione a operação apropriada *join* para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:
 - a. Identifique a coluna que é a chave na tabela `flights`;
 - b. Identifique a coluna que é a chave na tabela `airports`;
 - c. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?
 - d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
 - e. Armazene a tabela resultante no objeto `flights`.

tabela:

```
flights<- flights |> left_join(airports, by= c("DESTINATION_AIRPORT"= "IATA_CODE"))
flights
```

A tibble: 1,874 x 7

	DESTINATION_AIRPORT	soma	n_cia	CITY	STATE	LATITUDE	LONGITUDE
	<chr>	<dbl>	<int>	<chr>	<chr>	<dbl>	<dbl>
1	ABE	3413	350	Allentown	PA	40.7	-75.4
2	ABI	5926	446	Abilene	TX	32.4	-99.7
3	ABQ	20448	3241	Albuquerque	NM	35.0	-107.
4	ABR	-645	124	Aberdeen	SD	45.4	-98.4
5	ABY	1339	169	Albany	GA	31.5	-84.2
6	ACT	4436	271	Waco	TX	31.6	-97.2
7	ACV	2748	294	Arcata/Eureka	CA	41.0	-124.
8	ACY	9069	701	Atlantic City	NJ	39.5	-74.6
9	ADK	-293	19	Adak	AK	51.9	-177.
10	ADQ	72	59	Kodiak	AK	57.7	-152.

i 1,864 more rows

item c:

```
flights |>
  anti_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

A tibble: 0 x 7

i 7 variables: DESTINATION_AIRPORT <chr>, soma <dbl>, n_cia <int>,

CITY <chr>, STATE <chr>, LATITUDE <dbl>, LONGITUDE <dbl>

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
airports |>
  group_by(STATE)|>
  summarise(`AIRPORTS_BY_STATE` = n()) |>
  arrange(desc(`AIRPORTS_BY_STATE`))
```

A tibble: 54 x 2

STATE	AIRPORTS_BY_STATE
<chr>	<int>

```

1 TX                24
2 CA                22
3 AK                19
4 FL                17
5 MI                15
6 NY                14
7 CO                10
8 MN                8
9 MT                8
10 NC               8
# i 44 more rows

```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

1. Carregue o pacote `leaflet`;
2. Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
3. Armazene o grafico em b) numa variável chamada `this`;
4. Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

```

flights_map <- flights_mean |>
  left_join(
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) |>
  drop_na(LATITUDE, LONGITUDE)

library(leaflet)

this <- leaflet(flights_map) |>
  addTiles() |>
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~MEAN_DELAY
  )

this

```

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-29 14:30:10 -03"
```